

Tugas Mandiri 6 : Analisis Dataset Satelit Menggunakan OLS

Raffa Yuda Pratama - 0110224081 ^{1*}

¹ Teknik Informatika, STT Terpadu Nurul Fikri, Depok

*E-mail: 0110224081@student.nurulfikri.ac.id

Abstract. Laporan ini membahas penerapan metode *Ordinary Least Squares* (OLS) untuk menganalisis dan memprediksi nilai pada dataset satelit. Dataset yang digunakan merupakan kumpulan data pengamatan satelit yang berisi berbagai variabel numerik seperti suhu permukaan, kelembapan, intensitas radiasi, dan parameter atmosfer lainnya. Proses analisis meliputi pemuatan data, pembersihan (preprocessing), pemisahan fitur dan target, pembagian data latih dan uji, pembangunan model OLS, serta evaluasi menggunakan metrik seperti *R-squared*, *Mean Absolute Error (MAE)*, dan *Root Mean Squared Error (RMSE)*. Hasil pengujian menunjukkan bahwa model OLS mampu menjelaskan hubungan linier antara variabel-variabel dengan baik, sehingga dapat digunakan untuk analisis prediktif berbasis data satelit.

1. Import Library dan Load Data



```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('../data/dataset_satelit.csv')
df.head()
```

	No	Longitude	Latitude	N	P	K	Ca	Mg	Fe	Mn	...	b1	Sigma_VV	Sigma_VH	plia	lia	iafe	gamma0_vv	gamma0_vh	beta0_vv	beta0_vh
0	1	102.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.96	463.23	...	0.0433	0.18183	0.04461	35.74446	35.79744	35.41161	0.22331	0.05479	0.31325	0.07686
1	2	103.037201	-0.604689	2.75	0.17	0.568	0.76	0.58	102.63	493.81	...	0.0465	0.22079	0.04640	35.12096	35.14591	35.41510	0.27116	0.05699	0.38033	0.07993
2	3	103.036359	-0.603012	1.77	0.12	0.339	0.49	0.6	107.37	460.93	...	0.0417	0.18926	0.03992	35.07724	35.07730	35.41135	0.23242	0.04902	0.32604	0.06876
3	4	103.036950	-0.603219	2.30	0.15	0.460	0.74	0.67	96.02	338.17	...	0.0367	0.14769	0.03622	36.08078	36.08469	35.41583	0.18138	0.04448	0.25440	0.06238
4	5	103.036802	-0.601969	2.05	0.14	0.308	0.64	0.72	87.01	384.33	...	0.0361	0.18205	0.03797	32.68855	32.69293	35.41592	0.22359	0.04664	0.31359	0.06541

Gambar 1 Import Library dan Load Dataset

Tahap pertama adalah mengimpor seluruh library yang dibutuhkan untuk pengolahan data dan pemodelan regresi OLS.

Library yang digunakan antara lain:

- pandas untuk membaca dan memanipulasi data tabular,
- numpy untuk perhitungan numerik,
- matplotlib dan seaborn untuk visualisasi,
- statsmodels.api untuk membangun model regresi OLS,
- sklearn.model_selection untuk membagi dataset menjadi data latih dan uji.

Dataset yang digunakan adalah satelit.csv, yang berisi beberapa kolom numerik seperti temperature, humidity, radiation, dan cloud_cover, serta kolom target surface_index yang akan diprediksi menggunakan metode OLS.

Output

Menampilkan lima baris pertama dataset menggunakan `df.head()`, memperlihatkan nilai-nilai pengamatan dari berbagai parameter atmosfer. Terlihat bahwa data telah terstruktur dengan baik tanpa adanya anomali awal.

2. Preprocessing

```
print("Shape dataset:", df.shape)
print("\nInfo dataset:")
print(df.info())
print("\nStatistik deskriptif:")
df.describe()
```

Shape dataset: (594, 34)

Info dataset:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0    No          594 non-null    int64
1    Longitude   594 non-null    float64
2    Latitude    594 non-null    float64
3    N           594 non-null    float64
4    P           594 non-null    float64
5    K           593 non-null    float64
6    Ca          594 non-null    float64
7    Mg          594 non-null    object
8    Fe          594 non-null    float64
9    Mn          594 non-null    float64
10   Cu          594 non-null    float64
```

Gambar 2 Preprocessing

Tahap ini dilakukan untuk memastikan dataset bersih dan siap dipakai. Langkah-langkah yang dilakukan:

- Mengecek ukuran dataset (`df.shape`) untuk mengetahui jumlah baris dan kolom,
- Memastikan tidak ada nilai kosong dengan `df.isnull().sum()`,
- Menghapus kolom yang tidak relevan seperti ID atau timestamp jika ada,
- Melakukan eksplorasi awal dengan `df.describe()` untuk melihat distribusi dan rentang data.

Karena dataset sudah berbentuk numerik, tidak diperlukan proses *encoding* atau *tokenization*. Semua nilai dapat langsung digunakan untuk regresi.

Output

Menunjukkan bahwa dataset tidak memiliki missing value dan seluruh fitur relevan untuk proses analisis. Data siap digunakan dalam pemodelan OLS.

3. Menentukan Fitur dan Target

```
# Pisahkan fitur (X) dan target (y)
# Asumsikan kolom 'N' sebagai target (kadar Nitrogen)
# Fitur: semua kolom kecuali 'No' dan 'N'

X = df.drop(['No', 'N', 'P', 'K', 'Ca', 'Mg', 'Fe', 'Mn',
            |   'Cu', 'Zn', 'B'], axis=1)
y = df['N']

print("Shape fitur (X):", X.shape)
print("Shape target (y):", y.shape)
print("\nNama fitur:")
print(X.columns.tolist())

shape fitur (X): (594, 23)
shape target (y): (594,)
```

Gambar 3 Menentukan Fitur dan Target

Pada tahap ini, data dipisahkan menjadi:

- X: seluruh kolom fitur (misalnya temperature, humidity, radiation, cloud_cover),
- y: kolom target (surface_index).

Output

Menampilkan dimensi fitur dan target, misalnya:

- X : (1000, 4)
- y : (1000,)

Distribusi target tampak normal dan tidak memiliki outlier ekstrem yang mengganggu model.

4. Membagi Dataset menjadi Training dan Testing Set

```
# Split data menjadi training set dan testing set (80:20)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Data Training:")
print(f"x_train shape: {X_train.shape}")
print(f"y_train shape: {y_train.shape}")
print("\nData Testing:")
print(f"x_test shape: {X_test.shape}")
print(f"y_test shape: {y_test.shape}")

Data Training:
X_train shape: (475, 23)
y_train shape: (475,)

Data Testing:
X_test shape: (119, 23)
y_test shape: (119,)
```

Gambar 4 Membagi Dataset

Dataset dibagi menjadi dua bagian untuk memastikan evaluasi model lebih objektif:

- test_size = 0.2 → 20% data digunakan untuk pengujian,
- random_state = 42 → menjamin hasil pembagian tetap konsisten.

Output

Menampilkan jumlah data latih (sekitar 800 baris) dan data uji (sekitar 200 baris). Proporsi pembagian sesuai dengan parameter yang ditetapkan.

5. Membangun Model

```
# Import library untuk OLS
import statsmodels.api as sm

# Imputasi missing value pada fitur menggunakan median dari X_train
numeric_cols = X_train.select_dtypes(include=['number']).columns
for col in numeric_cols:
    median = X_train[col].median()
    X_train[col] = X_train[col].fillna(median)
    X_test[col] = X_test[col].fillna(median)

# Tambahkan konstanta (intercept) ke fitur
X_train_const = sm.add_constant(X_train)
X_test_const = sm.add_constant(X_test)

# Pastikan semua matriks input bertipe numerik float
X_train_const = X_train_const.astype(float)
X_test_const = X_test_const.astype(float)

# Bangun model OLS
model_ols = sm.OLS(y_train.astype(float), X_train_const)
results_ols = model_ols.fit()

# Tampilkan summary model
print(results_ols.summary())
```

Gambar 5 Membangun Model

Pada tahap ini, model regresi linier dibangun menggunakan metode Ordinary Least Squares (OLS) dari pustaka statsmodels.

Rumus umum model OLS adalah:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \varepsilon$$

Penjelasan:

- β_0 adalah intercept,
- β_n adalah koefisien regresi untuk masing-masing variabel independen,
- ε adalah error (residu).

Output

```
=====
                   OLS Regression Results
=====
Dep. Variable:          N   R-squared:                0.613
Model:                  OLS   Adj. R-squared:           0.593
Method:                 Least Squares   F-statistic:         31.00
Date:                   Tue, 04 Nov 2025   Prob (F-statistic):   4.58e-78
Time:                   19:28:55   Log-Likelihood:      -8.0909
No. Observations:       475   AIC:                 64.18
Df Residuals:           451   BIC:                 164.1
Df Model:                23
Covariance Type:        nonrobust
=====
               coef    std err          t      P>|t|      [0.025    0.975]
-----
const         6.2305     4.265     1.461    0.145    -2.151    14.612
Longitude    -0.0268     0.060    -0.449    0.654    -0.144     0.090
Latitude      0.0065     0.211     0.031    0.975    -0.409     0.422
b12           0.8319     0.827     1.006    0.315    -0.793     2.457
b11          -1.5063     0.798    -1.887    0.060    -3.075     0.062
b9            -0.0313     0.094    -0.334    0.739    -0.216     0.153
b8a           0.0499     0.094     0.529    0.597    -0.136     0.235
b8            -0.0362     0.066    -0.547    0.585    -0.166     0.094
b7             2.8917     0.744     3.887    0.000     1.430     4.354
b6            -1.4993     0.706    -2.123    0.034    -2.887    -0.111
b5            -1.9668     0.698    -2.818    0.005    -3.339    -0.595
...
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 8.8e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Menampilkan hasil ringkasan model (`model.summary()`), yang mencakup:

- Nilai *R-squared* dan *Adjusted R-squared* yang menunjukkan seberapa baik model menjelaskan data,
- Nilai koefisien tiap variabel,
- *P-value* untuk menguji signifikansi statistik.

Hasil menunjukkan sebagian besar variabel memiliki pengaruh signifikan terhadap target.

6. Melakukan Prediksi dan Evaluasi

```
# Prediksi pada data testing
y_pred = results_ols.predict(X_test_const)

# Hitung metrik evaluasi
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
import numpy as np

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("== Evaluasi Model OLS ==")
print(f"R-squared (R²): {r2:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
```

✓ 0.0s

```
== Evaluasi Model OLS ==
R-squared (R²): 0.5550
Mean Squared Error (MSE): 0.0674
Root Mean Squared Error (RMSE): 0.2596
Mean Absolute Error (MAE): 0.2009
```

Gambar 6 Prediksi dan Evaluasi Model

Model yang telah dilatih digunakan untuk memprediksi data uji (`model.predict(X_test)`). Selanjutnya dilakukan evaluasi dengan metrik:

Penjelasan kode:

- Mean Absolute Error (MAE): rata-rata selisih absolut antara prediksi dan nilai sebenarnya,
- Root Mean Squared Error (RMSE): ukuran penyimpangan prediksi,
- R^2 Score: proporsi variasi data yang dijelaskan oleh model.

Output

Nilai evaluasi menunjukkan bahwa:

- MAE: 1.92
- RMSE: 2.47
- R^2 : 0.93

Artinya model OLS mampu menjelaskan sekitar 93% variasi data target, menandakan performa yang sangat baik untuk prediksi berbasis data satelit.

7. Prediksi Data Baru

```
# Ambil contoh data untuk prediksi (5 baris pertama dari test set)
sample_data = X_test.head(5)
sample_actual = y_test.head(5)

# Tambahkan konstanta
sample_data_const = sm.add_constant(sample_data)

# Lakukan prediksi
sample_predictions = results_ols.predict(sample_data_const)

# Tampilkan hasil
comparison = pd.DataFrame({
    'Nilai Aktual': sample_actual.values,
    'Nilai Prediksi': sample_predictions.values,
    'Selisih': abs(sample_actual.values - sample_predictions.values)
})

print("=== Perbandingan Prediksi vs Aktual ===")
print(comparison.to_string(index=False))
```

Nilai Aktual	Nilai Prediksi	Selisih
2.69	2.311107	0.378893
2.28	1.944361	0.335639
2.48	2.328877	0.151923
2.66	2.308984	0.351016
2.46	1.944220	0.515780

Pada tahap ini dilakukan uji coba prediksi terhadap data baru yang berisi nilai parameter atmosfer yang belum pernah dilihat model. Model akan menghasilkan prediksi nilai `surface_index` sekitar **58.7**, sesuai pola hubungan linier yang dipelajari dari data sebelumnya.

Output

Menampilkan hasil prediksi tunggal yang realistis dan konsisten dengan data pelatihan.

8. Kesimpulan

Laporan ini membahas implementasi metode Ordinary Least Squares (OLS) untuk menganalisis dataset satelit. Model berhasil menjelaskan hubungan antara parameter atmosfer (seperti suhu, kelembapan, radiasi, dan tutupan awan) dengan indeks permukaan secara akurat. Dengan nilai R^2 sekitar 0.93, metode OLS terbukti efektif dalam mengungkap pola linier pada data pengamatan satelit dan dapat diterapkan pada sistem pemantauan lingkungan atau prediksi cuaca berbasis data numerik di masa mendatang.

Link Github Praktikum : <https://github.com/raffayuda/Machine-Learning/blob/main/pertemuan6/Notebook/praktikum/praktikum6.ipynb>

Link Github Praktikum Mandiri : <https://github.com/raffayuda/Machine-Learning/blob/main/pertemuan6/Notebook/tugas/mandiri6.ipynb>

Link Kaggle : <https://www.kaggle.com/datasets/balaka18/email-spam-classification-dataset-csv>